CI/CD Pipeline for Node.js Retail Application

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Shubham Kumawat	EN22CS301943
2.	Shivanshu Verma	EN22CS301917
3.	Shivani Vishwakarma	EN22CS301913
4.	Shubh Kapadia	EN22CS301942
5.	Shobha Goswami	EN22CS301922

Group Name: Group 06D8

Project Number: DO-41

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Avnesh Joshi

Academic Year: 2026

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Key outcomes*

Conclusion

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement

In traditional software development, the process of building, testing, and deploying applications is often done manually. This leads to several challenges such as delays in deployment, inconsistency across environments, and increased chances of human errors.

There is a need for an automated system that can streamline the software delivery process, ensure continuous integration of code changes, and enable fast and reliable deployment.

To address these challenges, a **CI/CD pipeline** is required that automates the entire workflow from code commit to production deployment, ensuring efficiency, consistency, and scalability.

2. Project Objectives

The main objective of this project is to design and implement an automated **CI/CD pipeline** for a Node.js Retail Application to improve efficiency, reliability, and deployment speed.

The specific objectives are:

- To automate the process of **code integration, build, testing, and deployment**
- To implement **Continuous Integration (CI)** for validating code changes
- To implement **Continuous Deployment (CD)** for automatic application deployment
- To containerize the application using **Docker** for consistency across environments
- To deploy the application on **AWS EC2** using cloud infrastructure
- To reduce manual errors and improve deployment accuracy
- To ensure scalability and maintainability of the application

3. Scope of the Project

The scope of this project is focused on designing and implementing a **CI/CD pipeline** for a Node.js-based Retail Application to automate the software development and deployment process.

The project includes:

- Automation of the **build, test, and deployment pipeline**
- Integration with **GitHub** for version control and code management
- Use of **CI/CD tools (GitHub Actions / Jenkins)** for pipeline execution
- Containerization of the application using **Docker**
- Storage of Docker images in **AWS ECR**
- Deployment of the application on **AWS EC2**

- Implementation of a **stateless and scalable architecture**

Proposed Solution

1. Key Features

- Automated **CI/CD pipeline** for build, test, and deployment
- Continuous integration with **automatic code validation**
- Continuous deployment to production environment
- **Docker-based containerization** for consistent environments
- Cloud deployment using **AWS EC2 and AWS ECR**
- **Stateless application design** for scalability
- Easy rollback using **versioned Docker images**
- Reduced manual intervention and human errors
- Fast and reliable application delivery

2. Overall Architecture / Workflow

The system follows a **CI/CD-based architecture** that automates the entire software delivery lifecycle from development to deployment.

Workflow Description

- Developer writes code and pushes it to GitHub
- Jenkins detects the changes and triggers the pipeline
- Jenkins builds a Docker image of the application
- The image is pushed to AWS ECR
- Kubernetes cluster running on EC2 pulls the image
- Application is deployed as pods in Kubernetes
- A Kubernetes service exposes the application using NodePort
- Users access the application through browser using Public IP and port 30080

3. Tools & Technologies Used

The project uses a combination of modern technologies and tools to implement the CI/CD pipeline and deploy the application efficiently.

- Git & GitHub - Used for version control and code repository
- Jenkins - Used for automating CI/CD pipeline
- Docker - Used to containerize the application
- AWS EC2 - Used to host the application and Kubernetes cluster
- AWS ECR - Used to store Docker images
- Kubernetes (k3s) - Used for container orchestration

- Node.js- Backend runtime environment
- React- Frontend user interface

Project Structure

The project follows a modular structure as described below:

- .github/ → Contains GitHub configurations
- dist/ → Production build files
- k8s/ → Kubernetes YAML configuration files
- node_modules/ → Installed dependencies
- public/ → Static assets
- src/ → Application source code
- Dockerfile → Defines Docker image build process
- Jenkinsfile → Defines CI/CD pipeline stages
- package.json → Project dependencies and scripts
- .env → Environment variables

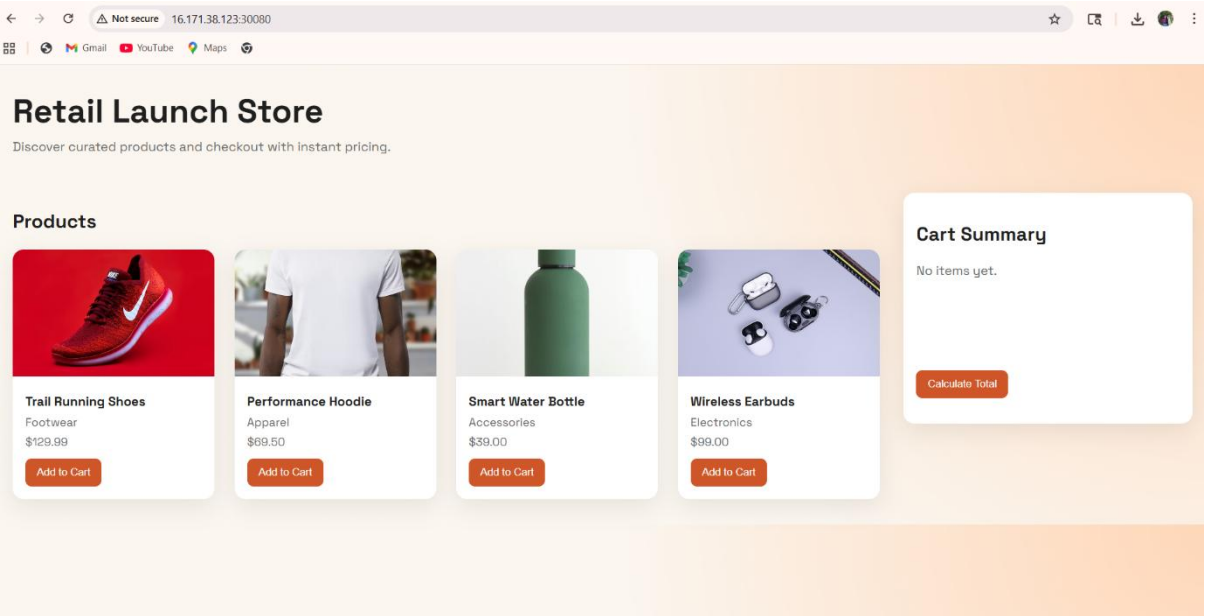
Results & Output

Application Deployment

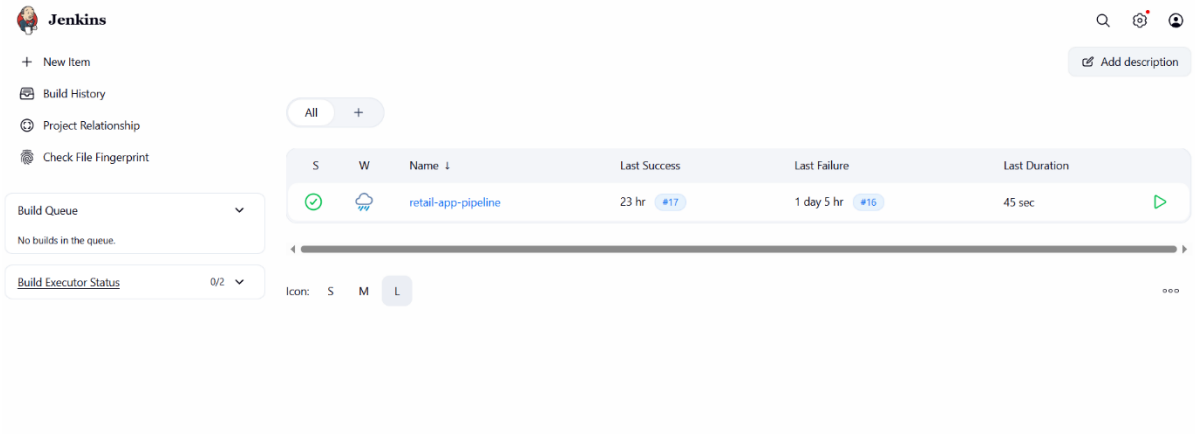
The application was successfully deployed on Kubernetes and is accessible via browser using: http://<public-ip>:30080. This confirms that the service is correctly exposing the application.

1. Screenshots / outputs

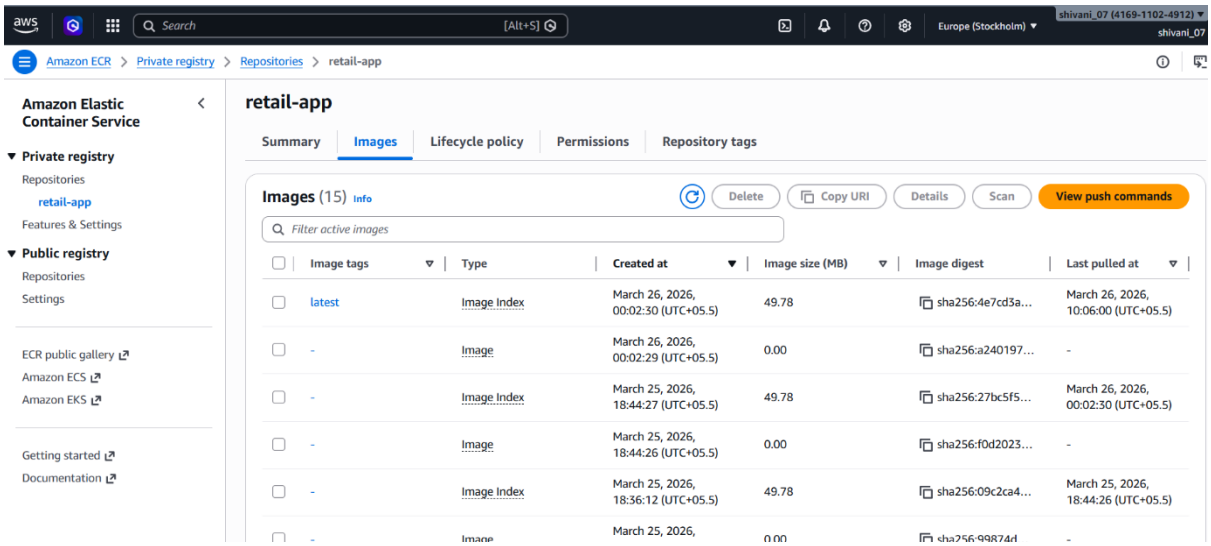
a. Application UI



b. Jenkins Pipeline



c. AWS ECR Repository



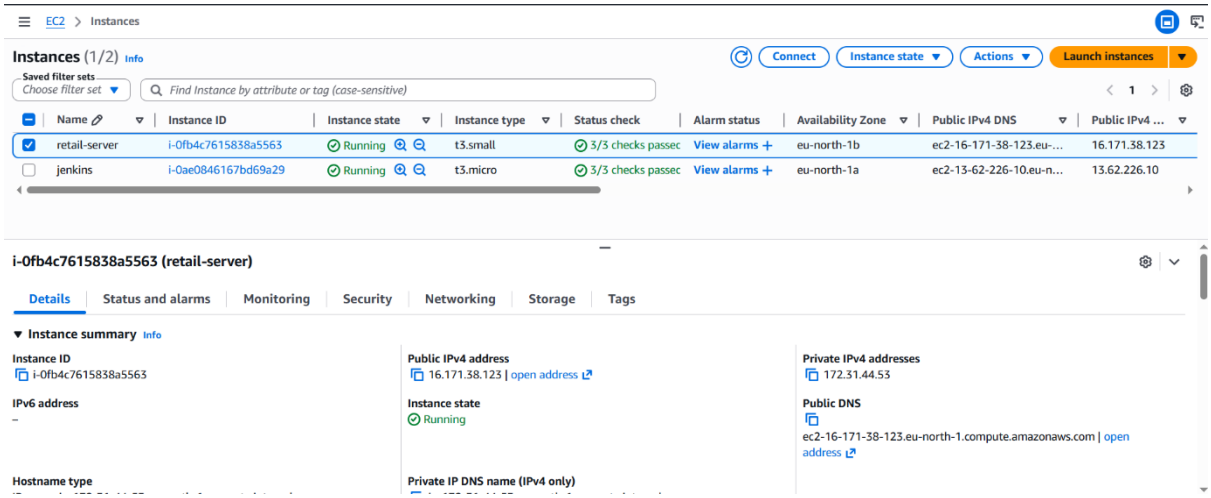
d. Kubernetes Pods

```
       _/m/'
Last login: Thu Mar 26 18:35:00 2026 from 106.222.215.97
[ec2-user@ip-172-31-44-53 ~]$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
retail-app-d44cfd65c-fqnv5    1/1     Running   0          13h
retail-app-d44cfd65c-qjvqh    1/1     Running   0          13h
[ec2-user@ip-172-31-44-53 ~]$
```

e. Kubernetes Service

```
[ec2-user@ip-172-31-44-53 ~]$
[ec2-user@ip-172-31-44-53 ~]$ kubectl get svc
NAME                 TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
kubernetes           ClusterIP   10.43.0.1     <none>        443/TCP        22h
retail-app-service   NodePort    10.43.16.51   <none>        80:30080/TCP   21h
[ec2-user@ip-172-31-44-53 ~]$
```

f. EC2 Instance



3. Key Outcomes

- Successful implementation of an **automated CI/CD pipeline**
- Reduced manual effort in deployment process
- Faster and more reliable software delivery
- Consistent application environment using Docker
- Seamless integration of GitHub, Docker, and AWS services
- Improved efficiency in development and deployment lifecycle

Conclusion

This project successfully demonstrates the design and implementation of a **CI/CD pipeline** for a Node.js Retail Application. By integrating tools such as GitHub, Docker, and AWS, the system automates the entire process of building, testing, and deploying the application.

The use of containerization ensures consistency across environments, while cloud deployment provides scalability and reliability. The CI/CD pipeline reduces manual intervention, minimizes errors, and enables faster delivery of applications.

Overall, the project highlights the importance of **DevOps practices** in modern software development and provides a strong foundation for building scalable and efficient systems.

Future Scope & Enhancements

Although the current system successfully implements a CI/CD pipeline, several enhancements can be made to improve functionality, scalability, and performance:

- Integration of monitoring tools like Prometheus and Grafana
- Implementation of auto-scaling in Kubernetes
- Addition of automated testing in CI/CD pipeline
- Use of Helm for better Kubernetes management
- Enhancement of application UI and features